

CSE401: Numerical Analysis, Simulation and Modeling

Solution of Systems of Equations: Gauss elimination, Gauss–Jordan elimination,
& LU decomposition

Nafis Tahmid

July 6, 2026

Department of CSE, BUET

Why are we here?

- You have n equations and n unknowns. You want the unknowns.
- Substitution by hand works. . . for $n = 2$, on a good day, with coffee.
- For $n = 1000$, “just substitute” can not be taken seriously.
- So we need **algorithms**.

The problem

We want to solve a general set of n equations in n unknowns:

$$a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1$$

$$a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2$$

$$\vdots$$

$$a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = b_n$$

or, because writing all of that more than once is tedious,

$$[A] \{X\} = \{B\}.$$

Motto: make $[A]$ so simple that $\{X\}$ falls out.

Gauss Elimination

The big idea: two steps

Step 1 — Forward Elimination

Knock out unknowns one by one until the system is upper triangular: each equation has one fewer unknown than the last.

$$\begin{bmatrix} \blacksquare & \blacksquare & \blacksquare \\ \blacksquare & \blacksquare & \blacksquare \\ \blacksquare & \blacksquare & \blacksquare \end{bmatrix} \Rightarrow \begin{bmatrix} \blacksquare & \blacksquare & \blacksquare \\ 0 & \blacksquare & \blacksquare \\ 0 & 0 & \blacksquare \end{bmatrix}$$

Step 2 — Back Substitution

The last equation has one unknown. Solve it. Substitute upward. Repeat until everyone is known.

Key Point

Forward elimination makes the matrix lazy-friendly.
Back substitution collects the reward.

Forward elimination: seeing the matrix

Cast the n equations as an **augmented matrix** $[A \mid \mathbf{b}]$:

$$\text{multiplier } m_{21} = \frac{a_{21}}{a_{11}} \left\{ \begin{array}{ccccc|c} \boxed{a_{11}} & a_{12} & a_{13} & \cdots & a_{1n} & b_1 \\ \boxed{a_{21}} & a_{22} & a_{23} & \cdots & a_{2n} & b_2 \\ a_{31} & a_{32} & a_{33} & \cdots & a_{3n} & b_3 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \cdots & a_{nn} & b_n \end{array} \right\} \leftarrow \text{pivot row}$$

$R_2 \leftarrow R_2 - m_{21} \cdot R_1$

The multiplier $m_{21} = a_{21}/a_{11}$ is chosen so that $a_{21} - \frac{a_{21}}{\mathbf{a_{11}}} \cdot \mathbf{a_{11}} = 0$.

Every other entry in Row 2 updates the same way: $a'_{2j} = a_{2j} - m_{21} a_{1j}$, $b'_2 = b_2 - m_{21} b_1$.

Forward elimination: watch the zero appear

After $R_2 \leftarrow R_2 - m_{21} R_1$:

$$\left. \begin{array}{l} \text{same trick} \\ m_{i1} = \frac{a_{i1}}{a_{11}} \end{array} \right\} \left(\begin{array}{ccccc|c} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} & b_1 \\ 0 & a'_{22} & a'_{23} & \cdots & a'_{2n} & b'_2 \\ a_{31} & a_{32} & a_{33} & \cdots & a_{3n} & b_3 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \cdots & a_{nn} & b_n \end{array} \right)$$

Repeat for every row below the pivot ($i = 3, \dots, n$) (pivot is still Row 1):

$$R_i \leftarrow R_i - \frac{a_{i1}}{a_{11}} R_1 \quad \Rightarrow \quad \text{entire first column below } a_{11} \text{ becomes } 0.$$

That completes Step 1 of forward elimination. The first column is now “clean” below the diagonal.

Step 1 in action: clearing column 1

$$\left(\begin{array}{cccc|c} a_{11} & a_{12} & a_{13} & a_{14} & b_1 \\ 0 & a'_{22} & a'_{23} & a'_{24} & b'_2 \\ 0 & a'_{32} & a'_{33} & a'_{34} & b'_3 \\ 0 & a'_{42} & a'_{43} & a'_{44} & b'_4 \end{array} \right) \begin{array}{l} \text{pivot} \\ \\ \\ \end{array}$$

Multipliers:

$$m_{21} = \frac{a_{21}}{a_{11}}, \quad R_2 \leftarrow R_2 - m_{21} R_1$$

$$m_{31} = \frac{a_{31}}{a_{11}}, \quad R_3 \leftarrow R_3 - m_{31} R_1$$

$$m_{41} = \frac{a_{41}}{a_{11}}, \quad R_4 \leftarrow R_4 - m_{41} R_1$$

Column 1 cleared below pivot ✓

Step 2 in action: clearing column 2

$$\left(\begin{array}{cccc|c} a_{11} & a_{12} & a_{13} & a_{14} & b_1 \\ 0 & a'_{22} & a'_{23} & a'_{24} & b'_2 \\ 0 & 0 & a''_{33} & a''_{34} & b''_3 \\ 0 & 0 & a''_{43} & a''_{44} & b''_4 \end{array} \right) \text{pivot}$$

New pivot: a'_{22} (Row 2)

Multipliers:

$$m_{32} = \frac{a'_{32}}{a'_{22}}, \quad R_3 \leftarrow R_3 - m_{32} R_2$$

$$m_{42} = \frac{a'_{42}}{a'_{22}}, \quad R_4 \leftarrow R_4 - m_{42} R_2$$

Column 2 cleared below pivot ✓

Fewer rows to update each step!

Step 3: the last push to upper-triangular

a_{11}	a_{12}	a_{13}	a_{14}	b_1
0	a'_{22}	a'_{23}	a'_{24}	b'_2
0	0	a''_{33}	a''_{34}	b''_3
0	0	0	a'''_{44}	b'''_4

pivot

New pivot: a''_{33} (Row 3)

Multiplier:

$$m_{43} = \frac{a'_{43}}{a''_{33}}, \quad R_4 \leftarrow R_4 - m_{43} R_3$$

Column 3 cleared ✓

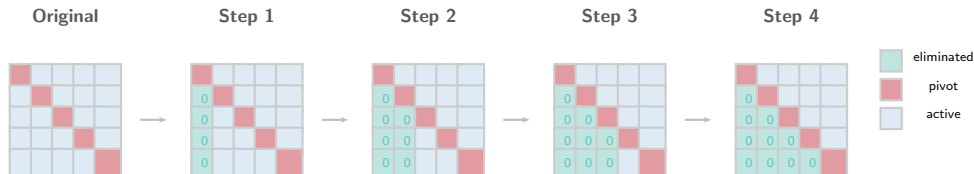
Upper-triangular!

$n-1 = 3$ steps total.

Ready for back substitution.

Forward elimination: marching down the diagonal

Repeat the trick with each successive row as pivot. The zero region grows like a staircase:



After $n - 1$ steps the system is **upper-triangular**:

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ \mathbf{0} & a'_{22} & a'_{23} & \cdots & a'_{2n} \\ \mathbf{0} & \mathbf{0} & a''_{33} & \cdots & a''_{3n} \\ \vdots & \vdots & & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \cdots & a_{nn}^{(n-1)} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b'_2 \\ b''_3 \\ \vdots \\ b_n^{(n-1)} \end{pmatrix}$$

Back substitution: unraveling from the bottom

Diagram illustrating the back substitution process for a system of linear equations. The matrix is shown in row echelon form, with the diagonal elements and the right-hand side values highlighted in green boxes. The rows are labeled x_1, x_2, x_3, x_4 from top to bottom. A green arrow labeled "solve order" points upwards, indicating the order of solving from the bottom row to the top row.

Last row — one unknown:

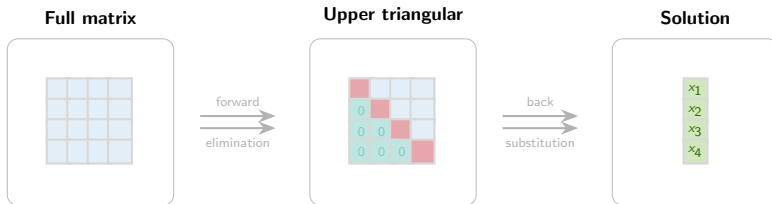
$$x_n = \frac{b_n^{(n-1)}}{a_{nn}^{(n-1)}}$$

Every row above ($i = n-1, \dots, 1$):

$$x_i = \frac{b_i^{(i-1)} - \sum_{j=i+1}^n a_{ij}^{(i-1)} x_j}{a_{ii}^{(i-1)}}$$

Every x_j in the sum is
already known from a later row.

The big picture: forward $\rightarrow$ triangular $\rightarrow$ back-solve



Example 1

The upward velocity of a rocket is measured at three times:

Time t (s)	Velocity v (m/s)
5	106.8
8	177.2
12	279.2

Fit $v(t) = a_1 t^2 + a_2 t + a_3$. Plugging in the three points gives

$$\begin{bmatrix} 25 & 5 & 1 \\ 64 & 8 & 1 \\ 144 & 12 & 1 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} 106.8 \\ 177.2 \\ 279.2 \end{bmatrix}.$$

Two steps of forward elimination, then back substitution. Let's go slowly.

Example 1 — forward elimination, first step

$$\left[\begin{array}{ccc|c} 25 & 5 & 1 & 106.8 \\ 64 & 8 & 1 & 177.2 \\ 144 & 12 & 1 & 279.2 \end{array} \right]$$

- Multiplier for row 2: $64/25 = 2.56$. $2.56 \times [25 \ 5 \ 1 \mid 106.8] = [64 \ 12.8 \ 2.56 \mid 273.408]$.
- Row 2 — that $\Rightarrow [0 \ -4.8 \ -1.56 \mid -96.208]$.
- Multiplier for row 3: $144/25 = 5.76$. $5.76 \times \text{Row 1} = [144 \ 28.8 \ 5.76 \mid 615.168]$.
- Row 3 — that $\Rightarrow [0 \ -16.8 \ -4.76 \mid -335.968]$.

$$\left[\begin{array}{ccc|c} 25 & 5 & 1 & 106.8 \\ 0 & -4.8 & -1.56 & -96.208 \\ 0 & -16.8 & -4.76 & -335.968 \end{array} \right]$$

Example 1 — forward elimination, second step

Pivot is now $a'_{22} = -4.8$. Multiplier for row 3: $-16.8 / -4.8 = 3.5$.

$$\left[\begin{array}{ccc|c} 25 & 5 & 1 & 106.8 \\ 0 & -4.8 & -1.56 & -96.208 \\ 0 & -16.8 & -4.76 & -335.968 \end{array} \right]$$

- $3.5 \times \text{Row 2} = [0 \ -16.8 \ -5.46 \ | \ -336.728]$.
- Row 3 – that $\Rightarrow [0 \ 0 \ 0.7 \ | \ 0.76]$.

$$\left[\begin{array}{ccc|c} 25 & 5 & 1 & 106.8 \\ 0 & -4.8 & -1.56 & -96.208 \\ 0 & 0 & 0.7 & 0.76 \end{array} \right]$$

Upper triangular. Forward elimination is done — two steps for three equations, as promised.

Example 1 — back substitution

$$\left[\begin{array}{ccc|c} 25 & 5 & 1 & 106.8 \\ 0 & -4.8 & -1.56 & -96.208 \\ 0 & 0 & 0.7 & 0.76 \end{array} \right]$$

- Bottom row: $0.7 a_3 = 0.76 \Rightarrow a_3 = 1.08571$.
- Middle row: $-4.8a_2 - 1.56a_3 = -96.208 \Rightarrow a_2 = 19.6905$.
- Top row: $25a_1 + 5a_2 + a_3 = 106.8 \Rightarrow a_1 = 0.290472$.

$$\{A\} = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} 0.290472 \\ 19.6905 \\ 1.08571 \end{bmatrix}$$

Example 1 — the payoff

The fitted polynomial:

$$v(t) = 0.290472 t^2 + 19.6905 t + 1.08571, \quad 5 \leq t \leq 12.$$

Evaluate wherever we like — e.g. by one matrix multiply:

$$\begin{bmatrix} v(6) \\ v(7.5) \\ v(9) \\ v(11) \end{bmatrix} = \begin{bmatrix} 129.686 \\ 165.104 \\ 201.828 \\ 252.828 \end{bmatrix} \text{ m/s.}$$

Three measurements in, a continuous velocity curve out. This is the whole reason we bothered.

So where's the catch? Pitfall 1: division by zero

Forward elimination divides by the pivot. If the pivot is **zero**, the method detonates immediately:

$$\begin{bmatrix} 0 & 5 & 6 \\ 4 & 5 & 7 \\ 9 & 2 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 11 \\ 16 \\ 15 \end{bmatrix}$$

The coefficient of x_1 in equation 1 is 0, there is nothing to pivot on. **Division by zero in step 1.**

The system is perfectly solvable. The method, taken naïvely, simply refuses.

Pitfall 1, the sneaky version: zeros that appear later

No zero pivot in sight at the start:

$$\begin{bmatrix} 5 & 6 & 7 \\ 10 & 12 & 3 \\ 20 & 17 & 19 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 18 \\ 25 \\ 56 \end{bmatrix}$$

Pivot is 5, all good. But after one step of forward elimination:

$$\begin{bmatrix} 5 & 6 & 7 \\ 0 & 0 & -11 \\ 0 & -7 & -9 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 18 \\ -11 \\ -16 \end{bmatrix}$$

The next pivot $a'_{22} = 0$. **Boom — division by zero in step 2.** *Moral: Never assume you're safe.*

Pitfall 2: round-off error (the quiet one)

Naïve Gauss elimination is prone to round-off, especially when:

- there are **many** equations, or
- we **subtract** nearly equal numbers.

We'll walk through the *same* system at two precisions — six and five significant digits — and watch the answer fall apart.

The system (exact answer: $x_1 = x_2 = x_3 = 1$)

$$\left(\begin{array}{ccc|c} 20 & 15 & 10 & 45 \\ -3 & -2.249 & 7 & 1.751 \\ 5 & 1 & 3 & 9 \end{array} \right)$$

Step 1 goes smoothly... but reveals a trap

Eliminate column 1 (multipliers -0.15 and 0.25):

$$\left(\begin{array}{ccc|c} 20 & 15 & 10 & 45 \\ 0 & \boxed{0.001} & 8.5 & 8.501 \\ 0 & -2.75 & 0.5 & -2.25 \end{array} \right) \rightarrow \begin{array}{l} \text{next pivot: } \mathbf{0.001} \\ \text{dividing by it = multiplying} \\ \text{by } \sim 1000 \end{array}$$

To clear column 2, we need the multiplier

$$m_{32} = \frac{-2.75}{\mathbf{0.001}} = -2750$$

Step 2: one lost digit changes everything

Multiply Row 2 by -2750 : $[0 \ -2.75 \ -23375] \ [-23377.75]$

Now **chop** the result to the working precision:

6 significant digits

$-23377.75 \rightarrow -23377.7$

Subtract from Row 3:

$$\left(\begin{array}{ccc|c} 20 & 15 & 10 & 45 \\ 0 & 0.001 & 8.5 & 8.501 \\ 0 & 0 & 23375.5 & -23375.4 \end{array} \right)$$

5 significant digits

$-23377.75 \rightarrow -23377$

Subtract from Row 3:

$$\left(\begin{array}{ccc|c} 20 & 15 & 10 & 45 \\ 0 & 0.001 & 8.5 & 8.501 \\ 0 & 0 & 23375 & -23374 \end{array} \right)$$

The last row's RHS differs by ≈ 1.4 . *Looks harmless? Watch what back substitution does...*

Back substitution: the error explodes

Solve x_3 from last row: $23375.5 x_3 = -23375.4$ vs. $23375 x_3 = -23374$
 $x_3 = 0.999995$ $x_3 = 0.99995$ (small error)

Solve x_2 : $0.001 x_2 = 8.501 - 8.5 x_3$
 $= 8.501 - 8.49995 = 0.00105$ vs. $= 8.501 - 8.4995 = 0.0015$
 $x_2 = 1.05$ $x_2 = 1.5$ (50% error!)

Solve x_1 : $20 x_1 = 45 - 15 x_2 - 10 x_3$
 $= 45 - 15.75 - 9.99995 = 19.250$ vs. $= 45 - 22.5 - 9.9995 = 12.500$
 $x_1 = 0.9625$ $x_1 = 0.625$ (37.5% error!)

error amplifies

close-ish

6 digits

$$\begin{bmatrix} 0.9625 \\ 1.05 \\ 0.999995 \end{bmatrix}$$

max error $\approx 5\%$

way off

5 digits

$$\begin{bmatrix} 0.625 \\ 1.5 \\ 0.99995 \end{bmatrix}$$

max error $\approx 50\%$

target

Exact

$$\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

One fewer digit of precision, an entirely wrong answer.

The culprit: dividing by the tiny pivot 0.001 turned a small rounding difference into a catastrophe.

Fixing round-off: more digits, or a smarter pivot

- Use double or quad precision. More digits, smaller round-off. *Helps with error...*
- ...**but** more precision does **nothing** for the division-by-zero pitfall. A zero pivot is still zero in 64 bits.
- **The real fix:** **partial pivoting** — attack both problems at once.

Gaussian elimination with partial pivoting

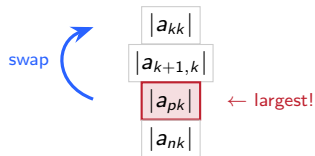
Same method as before, plus one rule at the **start of each step k** :

The pivoting rule

Among the candidate pivots in column k ,

$$|a_{kk}|, |a_{k+1,k}|, \dots, |a_{nk}|,$$

find the **largest**. If it sits in row p , **swap rows p and k** before eliminating.



Everything else — elimination and back substitution — is unchanged.

Cost: a little bookkeeping. Benefit: no zero pivots, and far less round-off.

Example 4 — partial pivoting, step 1

Same system that gave $x_1 = 0.625$ with naïve elimination. Now with pivoting, five significant digits:

$$\left(\begin{array}{ccc|c} 20 & 15 & 10 & 45 \\ -3 & -2.249 & 7 & 1.751 \\ 5 & 1 & 3 & 9 \end{array} \right) \begin{array}{l} |20| = 20 \leftarrow \text{largest} \\ |-3| = 3 \\ |5| = 5 \end{array}$$

Largest pivot is already in row 1. **No swap needed.** Eliminate as usual ($m_{21} = -0.15$, $m_{31} = 0.25$):

$$\left(\begin{array}{ccc|c} 20 & 15 & 10 & 45 \\ 0 & 0.001 & 8.5 & 8.501 \\ 0 & -2.75 & 0.5 & -2.25 \end{array} \right)$$

That 0.001 is the step-2 pivot. Last time we divided by it. This time...

Example 4 — the crucial swap

$$\left(\begin{array}{ccc|c} 20 & 15 & 10 & 45 \\ 0 & -2.75 & 0.5 & -2.25 \\ 0 & 0.001 & 8.5 & 8.501 \end{array} \right) \text{ safe pivot!}$$

Multiplier: $m_{32} = \frac{0.001}{-2.75} \approx -0.00036$ (naïve: -2750 . *Seven million times smaller!*) Eliminate:

$$\left(\begin{array}{ccc|c} 20 & 15 & 10 & 45 \\ 0 & -2.75 & 0.5 & -2.25 \\ 0 & 0 & 8.5001 & 8.5001 \end{array} \right)$$

Example 4 — back substitution, error gone

$$\left(\begin{array}{ccc|c} 20 & 15 & 10 & 45 \\ 0 & -2.75 & 0.5 & -2.25 \\ 0 & 0 & 8.5001 & 8.5001 \end{array} \right)$$

- $8.5001 x_3 = 8.5001 \Rightarrow x_3 = 1$
- $-2.75 x_2 + 0.5(1) = -2.25 \Rightarrow x_2 = 1$
- $20 x_1 + 15(1) + 10(1) = 45 \Rightarrow x_1 = 1$

With pivoting

$$\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

exact!

Naïve

$$\begin{bmatrix} 0.625 \\ 1.5 \\ 0.99995 \end{bmatrix}$$

50% off

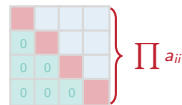
*Same five digits, same system.
Only difference: we swapped one row.*

Determinants for free

Forward elimination already does the hard work, so why not grab the determinant too?

Theorem 1

If $[B]$ comes from $[A]$ (a $n \times n$ matrix) by adding or subtracting a multiple of one row to another, $\det(A) = \det(B)$ (Same is true for column operations).



*Eliminate $\rightarrow$ triangular $\rightarrow$
multiply the diagonal.*

Theorem 2

If $[A]$ (a $n \times n$ matrix) is upper/lower triangular or diagonal,

$$\det(A) = \prod_{i=1}^n a_{ii}.$$

Example 5 — determinant by elimination

We already reduced the rocket matrix in Example 1:

$$\begin{array}{ccc} & [A] & \\ \left[\begin{array}{ccc} 25 & 5 & 1 \\ 64 & 8 & 1 \\ 144 & 12 & 1 \end{array} \right] & \xrightarrow{\text{eliminate}} & \begin{array}{ccc} & [B] & \\ \left[\begin{array}{ccc} \boxed{25} & 5 & 1 \\ 0 & \boxed{-4.8} & -1.56 \\ 0 & 0 & \boxed{0.7} \end{array} \right] & \end{array} \end{array}$$

By Theorem 1, $\det(A) = \det(B)$. By Theorem 2:

$$\det(A) = \mathbf{25} \times \mathbf{(-4.8)} \times \mathbf{0.7} = \mathbf{-84.00}.$$

When you had to pivot: mind the sign

Partial pivoting swaps rows. Swapping rows flips the determinant:

Theorem 3

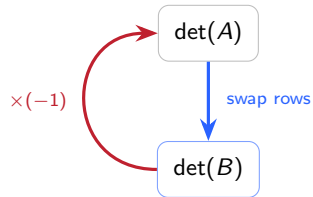
If $[B]$ results from $[A]$ by switching one row with another,

$$\det(B) = -\det(A).$$

So: **count the swaps**. Each one multiplies det by -1 .

Even swaps $\Rightarrow$ same sign.

Odd swaps $\Rightarrow$ flipped sign.



Example 6 — determinant with a row switch

A matrix whose pivoting forces **one** row switch:

$$\begin{array}{ccc} & [A] & \\ \left[\begin{array}{ccc} 5 & -1 & 5 \\ -3 & 2.099 & -6 \\ -10 & 7 & 0 \end{array} \right] & \xrightarrow[\text{pivoting}]{\text{partial}} & \begin{array}{ccc} & [B] & \\ \left[\begin{array}{ccc} \boxed{10} & -7 & 0 \\ 0 & \boxed{2.5} & 5 \\ 0 & 0 & \boxed{6.002} \end{array} \right] & \end{array} \end{array}$$

Theorem 2 on $[B]$: $\det(B) = \mathbf{10} \times \mathbf{2.5} \times \mathbf{6.002} = 150.05$. One row was switched $\Rightarrow$

Theorem 3:

$$\det(A) = -\det(B) = -\mathbf{150.05}.$$

Gauss–Jordan Elimination

One step further than Gauss elimination: drive the coefficient matrix to the **identity**.

Two upgrades over Gauss elimination

- When you eliminate an unknown, eliminate it from **every other** equation — not just the ones below.
- **Normalise** each row by its pivot, so every pivot becomes 1.

Consequence: the coefficient matrix becomes **identity**, not merely triangular. The right-hand side *is* the solution.

No back substitution required.

Gauss elimination

$$\begin{bmatrix} \blacksquare & \blacksquare & \blacksquare \\ 0 & \blacksquare & \blacksquare \\ 0 & 0 & \blacksquare \end{bmatrix}$$

*Upper triangular.
Then back-substitute.*

Gauss–Jordan

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

*Identity matrix.
Solution sits in the RHS.*

Gauss–Jordan does more arithmetic to avoid back substitution.

Example

$$3x_1 - 0.1x_2 - 0.2x_3 = 7.85$$

$$0.1x_1 + 7x_2 - 0.3x_3 = -19.3$$

$$0.3x_1 - 0.2x_2 + 10x_3 = 71.4$$

As an augmented matrix:

$$\left[\begin{array}{ccc|c} 3 & -0.1 & -0.2 & 7.85 \\ 0.1 & 7 & -0.3 & -19.3 \\ 0.3 & -0.2 & 10 & 71.4 \end{array} \right]$$

Goal: drive the left block to the identity, one pivot column at a time.

Normalise row 1 by its pivot 3:

$$\left[\begin{array}{ccc|c} 1 & -0.0333333 & -0.066667 & 2.61667 \\ 0.1 & 7 & -0.3 & -19.3 \\ 0.3 & -0.2 & 10 & 71.4 \end{array} \right]$$

Eliminate x_1 everywhere else: $R2 - 0.1 R1$ and $R3 - 0.3 R1$:

$$\left[\begin{array}{ccc|c} 1 & -0.0333333 & -0.066667 & 2.61667 \\ 0 & 7.00333 & -0.293333 & -19.5617 \\ 0 & -0.190000 & 10.0200 & 70.6150 \end{array} \right]$$

Normalise row 2 by its pivot 7.00333:

$$\left[\begin{array}{ccc|c} 1 & -0.0333333 & -0.066667 & 2.61667 \\ 0 & 1 & -0.0418848 & -2.79320 \\ 0 & -0.190000 & 10.0200 & 70.6150 \end{array} \right]$$

Eliminate x_2 from rows 1 and 3 (not just below!):

$$\left[\begin{array}{ccc|c} 1 & 0 & -0.0680629 & 2.52356 \\ 0 & 1 & -0.0418848 & -2.79320 \\ 0 & 0 & 10.01200 & 70.0843 \end{array} \right]$$

Notice row 1 just got cleaned up too. That's the difference with Gauss

Normalise row 3 by its pivot 10.0120:

$$\left[\begin{array}{ccc|c} 1 & 0 & -0.0680629 & 2.52356 \\ 0 & 1 & -0.0418848 & -2.79320 \\ 0 & 0 & 1 & 7.0000 \end{array} \right]$$

Eliminate x_3 from rows 1 and 2:

$$\left[\begin{array}{ccc|c} 1 & 0 & 0 & 3.0000 \\ 0 & 1 & 0 & -2.5000 \\ 0 & 0 & 1 & 7.0000 \end{array} \right]$$

Left block is the identity, so read off the answer: $x_1 = 3$, $x_2 = -2.5$, $x_3 = 7$.

No back substitution was ever needed.

Gauss–Jordan — the fine print

- Every pitfall and improvement from Gauss elimination **also** applies here: division by zero, round-off, and the cure — **partial pivoting** — carry over unchanged.
- No back substitution.

LU Decomposition

The idea: split the matrix once, reuse forever

If Naïve Gauss forward elimination works on $[A]$, we can always write

$$[A] = [L][U],$$

$[L]$ = lower triangular, $[U]$ = upper triangular.

To solve $[A][X] = [C]$, substitute $[A] = [L][U]$:

$$[L][U][X] = [C].$$

Multiply by $[L]^{-1}$ and let $[Z] = [U][X]$. The single hard problem splits into two easy triangular ones:

$$\underbrace{[L][Z] = [C]}_{\text{forward substitution}} \quad \text{then} \quad \underbrace{[U][X] = [Z]}_{\text{back substitution}}.$$

What is $[U]$? You already know

Run Naïve Gauss forward elimination on $[A]$. The upper-triangular matrix sitting in front of you at the end is $[U]$.

$$[U] = \begin{bmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & \cdots & u_{2n} \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & u_{nn} \end{bmatrix}$$

Nothing new to compute. You did this in the Gauss elimination.

What exactly is a multiplier?

Recall how forward elimination zeroes out entries below the pivot.

In **step k** , the pivot is $a_{kk}^{(k-1)}$. To zero out $a_{ik}^{(k-1)}$ (row i , column k), we compute

$$m_{ik} = \frac{a_{ik}^{(k-1)}}{a_{kk}^{(k-1)}}, \quad i = k + 1, k + 2, \dots, n.$$

Then we update every entry in row i :

$$a_{ij}^{(k)} = a_{ij}^{(k-1)} - m_{ik} \cdot a_{kj}^{(k-1)}, \quad j = k + 1, \dots, n.$$

The number m_{ik} is the **multiplier** that was *just the right factor* to kill position (i, k) .

In Gauss you used these multipliers and moved on. LU says: write them down.

Building $[L]$: where the multipliers live

$[L]$ is a lower-triangular matrix with **ones on the diagonal** and the multiplier m_{ij} in position (i, j) :

$$[L] = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ m_{21} & 1 & 0 & \cdots & 0 \\ m_{31} & m_{32} & 1 & & \vdots \\ \vdots & & & \ddots & 0 \\ m_{n1} & m_{n2} & \cdots & m_{n,n-1} & 1 \end{bmatrix}$$

Why position (i, j) ? Because m_{ij} is the factor that was used in step j to zero out entry (i, j) of the matrix.

Key insight

Forward elimination was secretly building $[L]$ the whole time. You just had to write the multipliers down instead of discarding them.

The LU procedure at a glance

1. Start with $[A]$.
2. Perform Naïve Gauss forward elimination. At each step k :
 - Compute multipliers $m_{ik} = a_{ik}/a_{kk}$, $i = k+1, \dots, n$.
 - Store m_{ik} in position (i, k) of $[L]$.
 - Update the remaining rows: $a_{ij} \leftarrow a_{ij} - m_{ik} a_{kj}$.
3. At the end, the transformed matrix is $[U]$, and $[L]$ (with ones on the diagonal) is the collection of all stored multipliers.

Same work as Gauss. The only extra effort is writing down numbers you were already computing.

Example 1 — LU of the rocket matrix, step 1

$$[A] = \begin{bmatrix} 25 & 5 & 1 \\ 64 & 8 & 1 \\ 144 & 12 & 1 \end{bmatrix}$$

Step 1 (pivot = $a_{11} = 25$, zero out column 1 below it):

- $m_{21} = \frac{a_{21}}{a_{11}} = \frac{64}{25} = 2.56$. Store in $[L]$ at position $(2, 1)$.
- $m_{31} = \frac{a_{31}}{a_{11}} = \frac{144}{25} = 5.76$. Store in $[L]$ at position $(3, 1)$.

Update rows 2 and 3 ($R_i \leftarrow R_i - m_{i1} R_1$):

$$[A] \rightarrow \begin{bmatrix} 25 & 5 & 1 \\ 0 & 8 - 2.56(5) & 1 - 2.56(1) \\ 0 & 12 - 5.76(5) & 1 - 5.76(1) \end{bmatrix} = \begin{bmatrix} 25 & 5 & 1 \\ 0 & -4.8 & -1.56 \\ 0 & -16.8 & -4.76 \end{bmatrix}$$

Example 1 — LU of the rocket matrix, step 2

Matrix after step 1:
$$\begin{bmatrix} 25 & 5 & 1 \\ 0 & -4.8 & -1.56 \\ 0 & -16.8 & -4.76 \end{bmatrix}$$

Step 2 (pivot = $a_{22}^{(2)} = -4.8$, zero out column 2 below it):

- $m_{32} = \frac{a_{32}^{(2)}}{a_{22}^{(2)}} = \frac{-16.8}{-4.8} = 3.5$. Store in $[L]$ at position $(3, 2)$.

Update row 3 ($R_3 \leftarrow R_3 - 3.5 R_2$):

$$a_{33}^{(3)} = -4.76 - 3.5(-1.56) = -4.76 + 5.46 = 0.7.$$

The matrix is now upper-triangular — that is $[U]$:

$$[U] = \begin{bmatrix} 25 & 5 & 1 \\ 0 & -4.8 & -1.56 \\ 0 & 0 & 0.7 \end{bmatrix}$$

Example 1 — assembling $[L]$ and $[U]$

Collect every multiplier into $[L]$ (ones on the diagonal, multiplier m_{ij} at position (i,j)):

$$[L] = \begin{bmatrix} 1 & 0 & 0 \\ 2.56 & 1 & 0 \\ 5.76 & 3.5 & 1 \end{bmatrix}, \quad [U] = \begin{bmatrix} 25 & 5 & 1 \\ 0 & -4.8 & -1.56 \\ 0 & 0 & 0.7 \end{bmatrix}$$

Reading the story back:

- $\ell_{21} = 2.56$: the factor that zeroed out position $(2, 1)$.
- $\ell_{31} = 5.76$: zeroed out $(3, 1)$.
- $\ell_{32} = 3.5$: zeroed out $(3, 2)$.

Each multiplier sits at exactly the position it killed. That is the entire pattern.

Example 1 — sanity check

$$[L][U] = \begin{bmatrix} 1 & 0 & 0 \\ 2.56 & 1 & 0 \\ 5.76 & 3.5 & 1 \end{bmatrix} \begin{bmatrix} 25 & 5 & 1 \\ 0 & -4.8 & -1.56 \\ 0 & 0 & 0.7 \end{bmatrix} = \begin{bmatrix} 25 & 5 & 1 \\ 64 & 8 & 1 \\ 144 & 12 & 1 \end{bmatrix} = [A].$$

It checks out. *Multiply it through by hand once in your life; you'll trust it forever after.*

Solving with LU — the two-step dance

Given $[A] = [L][U]$, solve $[A][X] = [C]$ in two passes.

1. **Forward substitution** on $[L][Z] = [C]$:

Solve top-to-bottom. Because $[L]$ has ones on the diagonal, no division is needed:

$$z_1 = c_1, \quad z_i = c_i - \sum_{j=1}^{i-1} \ell_{ij} z_j, \quad i = 2, \dots, n.$$

2. **Back substitution** on $[U][X] = [Z]$:

Solve bottom-to-top, exactly as in Gauss:

$$x_n = \frac{z_n}{u_{nn}}, \quad x_i = \frac{z_i - \sum_{j=i+1}^n u_{ij} x_j}{u_{ii}}, \quad i = n-1, \dots, 1.$$

Example 1 — forward substitution for $[Z]$

Solve with $[C] = [106.8, 177.2, 279.2]^T$. First $[L][Z] = [C]$:

$$\begin{bmatrix} 1 & 0 & 0 \\ 2.56 & 1 & 0 \\ 5.76 & 3.5 & 1 \end{bmatrix} \begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix} = \begin{bmatrix} 106.8 \\ 177.2 \\ 279.2 \end{bmatrix}$$

Forward substitution, top to bottom:

- $z_1 = 106.8$.
- $z_2 = 177.2 - 2.56(106.8) = 177.2 - 273.408 = -96.208$.
- $z_3 = 279.2 - 5.76(106.8) - 3.5(-96.208) = 279.2 - 615.168 + 336.728 = 0.76$.

$$[Z] = \begin{bmatrix} 106.8 \\ -96.208 \\ 0.76 \end{bmatrix}$$

Look familiar? This is exactly the RHS at the end of Gauss forward elimination. $[Z] = [L]^{-1}[C]$ is that modified RHS — not a coincidence.

Example 1 — back substitution for $[X]$

Now $[U][X] = [Z]$:

$$\begin{bmatrix} 25 & 5 & 1 \\ 0 & -4.8 & -1.56 \\ 0 & 0 & 0.7 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} 106.8 \\ -96.208 \\ 0.76 \end{bmatrix}$$

Back substitution, bottom to top:

- $0.7 a_3 = 0.76 \Rightarrow a_3 = 1.0857$.
- $-4.8 a_2 - 1.56(1.0857) = -96.208 \Rightarrow a_2 = 19.691$.
- $25 a_1 + 5(19.691) + 1.0857 = 106.8 \Rightarrow a_1 = 0.29048$.

Same answer as Gauss elimination — of course it is.

“But this looks like more work!”

A fair suspicion.

Gauss elimination does the whole job in one pass (forward elimination + back substitution).

LU splits it into *three* parts — decomposition, forward substitution, back substitution.

Surely three parts cost more than two?

Let's count clock cycles and find out.

How we count: the clock-cycle model

We assign a cost in **clock cycles** to each arithmetic operation. A commonly used model (AMD K7 chip):

addition (+)	:	4 clock cycles
subtraction (−)	:	4 clock cycles
multiplication (×)	:	4 clock cycles
division (÷)	:	16 clock cycles

If one clock cycle takes T seconds, then:

$$\text{add/sub/mul} = 4T, \quad \text{div} = 16T.$$

For a 1.2 GHz CPU, $T = 1/(1.2 \times 10^9) \approx 0.833 \text{ ns}$.

Decomposition cost: what happens in step k

First, we count the cost of *just the decomposition* — computing $[L]$ and $[U]$.

In step k of forward elimination on $[A]$ alone (no RHS yet):

- Rows to update: $i = k+1, \dots, n \Rightarrow (n - k)$ rows.
- For each such row i :
 - Compute multiplier $m_{ik} = a_{ik}/a_{kk}$: 1 division $\rightarrow 16T$.
 - For each column $j = k+1, \dots, n$ ($n - k$ columns, we can just put 0 in column k):
 - $a_{ij} \leftarrow a_{ij} - m_{ik} \cdot a_{kj}$: 1 mul + 1 sub $\rightarrow 4T + 4T = 8T$.

Cost for one row in step k :

$$16T + (n - k) \cdot 8T = [16 + 8(n - k)] T.$$

Cost for the entire step k ($n - k$ rows):

$$(n - k)[16 + 8(n - k)] T$$

Decomposition cost: summing over all steps

Total decomposition cost (computing l_{ij} and u_{ij}) is the sum over all $n - 1$ steps:

$$CT|_{DE} = \sum_{k=1}^{n-1} (n-k) [16 + 8(n-k)] T.$$

Substitute $p = n - k$ (so $p = 1, 2, \dots, n-1$):

$$= \sum_{p=1}^{n-1} [16p + 8p^2] T = 16T \sum_{p=1}^{n-1} p + 8T \sum_{p=1}^{n-1} p^2.$$

Standard sum formulae:

$$\sum_{p=1}^{n-1} p = \frac{(n-1)n}{2}, \quad \sum_{p=1}^{n-1} p^2 = \frac{(n-1)n(2n-1)}{6}.$$

Substituting:

$$CT|_{DE} = 16T \cdot \frac{n(n-1)}{2} + 8T \cdot \frac{n(n-1)(2n-1)}{6}.$$

Decomposition cost: expanding to the final formula

$$CT|_{DE} = 8n(n-1) T + \frac{4n(n-1)(2n-1)}{3} T.$$

Expand each piece:

$$\begin{aligned} 8n(n-1) &= 8n^2 - 8n, \\ \frac{4n(n-1)(2n-1)}{3} &= \frac{4(2n^3 - 3n^2 + n)}{3} = \frac{8n^3 - 12n^2 + 4n}{3}. \end{aligned}$$

Combine over a common denominator of 3:

$$CT|_{DE} = \frac{8n^3 - 12n^2 + 4n + 24n^2 - 24n}{3} T = \frac{8n^3 + 12n^2 - 20n}{3} T.$$

$$\boxed{CT|_{DE} = \left(\frac{8}{3}n^3 + 4n^2 - \frac{20}{3}n \right) T}$$

Forward substitution cost

Solve $[L][Z] = [C]$ top-to-bottom. Row i :

$$z_i = c_i - \sum_{j=1}^{i-1} \ell_{ij} z_j.$$

- Row 1: $z_1 = c_1$ — free (no arithmetic).
- Row i ($i \geq 2$): $(i-1)$ multiplications + $(i-1)$ subtractions = $8(i-1) T$.
No division, because $\ell_{ii} = 1$.

Total:

$$CT|_{FS} = \sum_{i=2}^n 8(i-1) T = 8T \sum_{p=1}^{n-1} p = 8T \cdot \frac{n(n-1)}{2} = 4n(n-1) T.$$

$CT|_{FS} = (4n^2 - 4n) T$

Back substitution cost

Solve $[U][X] = [Z]$ bottom-to-top. Row i :

$$x_i = \frac{z_i - \sum_{j=i+1}^n u_{ij} x_j}{u_{ii}}.$$

- Row i : $(n - i)$ multiplications, $(n - i)$ subtractions, and **1 division**.

$$\text{Cost} = 8(n - i) T + 16T = [8(n - i) + 16] T.$$

The division by u_{ii} adds 16 per row.

- This holds for every row $i = 1, \dots, n$ (for row n : $n - i = 0$, cost = $16T$).

Total:

$$CT|_{BS} = \sum_{i=1}^n [8(n - i) + 16] T = 8T \sum_{p=0}^{n-1} p + 16nT = 8T \cdot \frac{n(n - 1)}{2} + 16nT.$$

$$= 4n(n - 1) T + 16nT = (4n^2 - 4n + 16n) T$$

$CT|_{BS} = (4n^2 + 12n) T$

Total LU cost for one solve

$$\begin{aligned}CT|_{LU} &= CT|_{DE} + CT|_{FS} + CT|_{BS} \\&= \left(\frac{8}{3}n^3 + 4n^2 - \frac{20}{3}n \right) T + (4n^2 - 4n) T + (4n^2 + 12n) T \\&= \left(\frac{8}{3}n^3 + \underbrace{(4 + 4 + 4)}_{12}n^2 + \underbrace{\left(-\frac{20}{3} - 4 + 12 \right)}_{\frac{4}{3}}n \right) T\end{aligned}$$

$$CT|_{LU} = \left(\frac{8}{3}n^3 + 12n^2 + \frac{4}{3}n \right) T$$

Now let's line it up against Gauss.

Gauss forward elimination transforms $[A]$ and the RHS simultaneously. Its forward-elimination cost is *larger* than $CT|_{DE}$ by exactly the cost of transforming the RHS (one multiplication and one subtraction per row per step, incurring $8T \times \sum_{i=1}^{n-1} n - i = (4n^2 - 4n)T$) :

$$CT|_{FE} = \left(\frac{8}{3}n^3 + 8n^2 - \frac{32}{3}n \right) T = CT|_{DE} + \underbrace{(4n^2 - 4n) T}_{\text{RHS work}}.$$

Back substitution is the same in both methods: $CT|_{BS} = (4n^2 + 12n) T$.

$$CT|_{GE} = CT|_{FE} + CT|_{BS} = \left(\frac{8}{3}n^3 + 12n^2 + \frac{4}{3}n \right) T.$$

Identical

$CT|_{LU} = CT|_{GE}$. For a *single* solve, LU costs exactly the same as Gauss. So far we have learned a new method that buys us... nothing.

Patience. The payoff appears the moment the RHS changes while $[A]$ stays put.

Where LU wins: same $[A]$, many right-hand sides

Finding $[A]^{-1}$ means solving $[A][X] = [C]$ for the n columns of the identity — n systems, **same** $[A]$.

LU

Decompose **once** ($\sim n^3$), then do forward + back substitution n times ($\sim n^2$ each, $\times n = n^3$):

Dominant term $\sim n^3$.

Gauss

No reusable decomposition — redo forward elimination **every** time ($\sim n^3$ each, $\times n = n^4$):

Dominant term $\sim n^4$.

This is the whole point of LU.

Inverse cost: the full derivation

LU: decompose once, forward-sub n times, back-sub n times.

$$\begin{aligned}CT|_{\text{inv},LU} &= 1 \times CT|_{DE} + n \times CT|_{FS} + n \times CT|_{BS} \\&= \left(\frac{8}{3}n^3 + 4n^2 - \frac{20}{3}n\right) T + n(4n^2 - 4n) T + n(4n^2 + 12n) T \\&= \left(\frac{8}{3}n^3 + 4n^3 + 4n^3 + 4n^2 - 4n^2 + 12n^2 - \frac{20}{3}n\right) T \\&= \left(\frac{32}{3}n^3 + 12n^2 - \frac{20}{3}n\right) T.\end{aligned}$$

Gauss: forward elimination n times, back-sub n times.

$$\begin{aligned}CT|_{\text{inv},GE} &= n \times CT|_{FE} + n \times CT|_{BS} \\&= \left(\frac{8}{3}n^4 + 12n^3 + \frac{4}{3}n^2\right) T.\end{aligned}$$

$$\text{Ratio for large } n: \frac{CT|_{\text{inv},GE}}{CT|_{\text{inv},LU}} \approx \frac{\frac{8}{3}n^4}{\frac{32}{3}n^3} = \frac{n}{4}.$$

LU vs. Gauss for the inverse — the numbers

n	10	100	1000	10000
$CT _{\text{inv},GE} / CT _{\text{inv},LU}$	3.28	25.83	250.8	2501

At $n = 10000$, Gauss does roughly **2500 times** the work to invert the same matrix.

Decompose once, reuse often. That is LU's entire career in one sentence.

Find inverse of $[A]$ using LU

$[B] = [A]^{-1}$ satisfies $[A][B] = [I]$. Column by column, the j -th column of $[B]$ solves

$$[A] \{b_{.j}\} = \{e_j\},$$

where $\{e_j\}$ is the j -th column of the identity.

Procedure:

1. Decompose $[A] = [L][U]$ — once.
2. For each $j = 1, 2, \dots, n$:
 - Forward-substitute $[L][Z] = [e_j]$ to get $[Z]$.
 - Back-substitute $[U]\{b_{.j}\} = [Z]$ to get column j of the inverse.

Finding inverse of $[A]$ – First column

Solve $[A]\{b_{.1}\} = [1, 0, 0]^T$ using the LU we already have.

Forward: $[L][Z] = [1, 0, 0]^T$

- $z_1 = 1$
- $z_2 = 0 - 2.56(1) = -2.56$
- $z_3 = 0 - 5.76(1) - 3.5(-2.56) = -5.76 + 8.96 = 3.2$

Back: $[U]\{b_{.1}\} = [1, -2.56, 3.2]^T$

- $0.7 b_{31} = 3.2 \Rightarrow b_{31} = 4.571$
- $-4.8 b_{21} - 1.56(4.571) = -2.56 \Rightarrow b_{21} = -0.9524$
- $25 b_{11} + 5(-0.9524) + 4.571 = 1 \Rightarrow b_{11} = 0.04762$

$$\{b_{.1}\} = \begin{bmatrix} 0.04762 \\ -0.9524 \\ 4.571 \end{bmatrix}$$

Finding inverse of $[A]$ – Second and third columns

Repeat with RHS $[0, 1, 0]^T$ and $[0, 0, 1]^T$ — same $[L]$, same $[U]$, only the RHS changes:

$$[A]^{-1} = \begin{bmatrix} 0.04762 & -0.08333 & 0.03571 \\ -0.9524 & 1.417 & -0.4643 \\ 4.571 & -5.000 & 1.429 \end{bmatrix}$$

Quick sanity check — pick any column of $[A]^{-1}[A]$ and verify it gives the corresponding identity column.

Thank You